

MILESTONE 1 REPORT: DATA ACQUISITION

Môn học: SEG301 - Search Engines & Information Retrieval

Nhóm: Birds Search Engine

Chủ đề: Topic 4 - Social Listening (Lắng nghe Mạng xã hội)

Nguồn dữ liệu chính: VOZ Forum (F17/F33)

Ngày nộp: 2026-01-25

1. TỔNG QUAN DỰ ÁN (Project Overview)

1.1 Kết quả đạt được

Dựa trên quá trình thu thập và xử lý dữ liệu, nhóm đã hoàn thành vượt mức các chỉ tiêu đề ra cho Milestone 1. Số liệu dưới đây được trích xuất từ file thống kê mới nhất (statistics_report.md):

Chỉ số	Giá trị đạt được	Yêu cầu	Trạng thái
Tổng số Documents	1,135,076	1,000,000	✓ Vượt 13.5%
Tổng số từ (Tokens)	154,972,519	-	-
Độ dài trung bình	136.53 từ/doc	> 50 từ	✓ Đạt chuẩn
Số lượng Thread	112,528	-	Đa dạng chủ đề
Số lượng Tác giả	31,063	-	Đa dạng quan điểm

1.2 Định nghĩa Document

Để phù hợp với bài toán **Social Listening**, nhóm định nghĩa một Document hợp lệ là:

1. Một bài viết (Post) hoặc Bình luận (Comment) thảo luận về các vấn đề xã hội/công nghệ.
2. **Độ dài tối thiểu:** 50 từ (sau khi đã tách từ và loại bỏ stopwords cơ bản).
3. **Ngôn ngữ:** Tiếng Việt (có hỗ trợ từ khóa tiếng Anh chuyên ngành).

2. QUY TRÌNH KỸ THUẬT (Technical Approach)

2.1 Kiến trúc Crawler

Nhóm chuyển đổi từ kiến trúc AsyncIO sang mô hình Multi-threading Worker Pool để đảm bảo khả năng vượt qua cơ chế chặn bot (Cloudflare) của VOZ, đồng thời duy trì hiệu suất cao khi xử lý hàng triệu requests.

a. Công nghệ cốt lõi (Core Stack):

- Engine: Python 3.x với `concurrent.futures.ThreadPoolExecutor`.
- Request Client: `cloudscraper` (thay vì `requests` thường) để giả lập browser fingerprint, tự động bypass các trang "Just a moment..." của Cloudflare.
- HTML Parser: `BeautifulSoup4` với parser `lxml` cho tốc độ phân tích cú pháp cực nhanh.
- Progress Tracking: `tqdm` để hiển thị tiến độ thời gian thực.

b. Cơ chế hoạt động (Concurrency Model): Hệ thống hoạt động theo mô hình

Producer-Consumer với Thread Pool:

- Scout Thread (Producer): Quét các trang danh mục (F17, F33, F34...) để thu thập danh sách Thread ID.
- Worker Threads (Consumers): Mặc định 20 workers hoạt động song song. Mỗi worker nhận một Thread, crawl toàn bộ các page trong thread đó, parse comment và trích xuất dữ liệu.
- Connection Pooling: Sử dụng `requests.adapters.HTTPAdapter` với `pool_connections=10`, `pool_maxsize=20` để tái sử dụng kết nối TCP, giảm độ trễ bắt tay SSL.

c. Tính năng Tin cậy & Ổn định (Reliability Features):

- Cơ chế Checkpoint (Resume Capability):
 - Sử dụng class `CrawlCheckpoint` để tuần tự hóa (serialize) trạng thái vào file `crawl_checkpoint.pkl`.
 - Lưu trữ: Tập `crawled_threads`, `crawled_posts`, và trang cuối cùng đang crawl của từng forum.
 - Cho phép dừng và chạy lại crawler bất cứ lúc nào mà không bị mất dữ liệu hay crawl lại từ đầu.
- Anti-Blocking Strategy:
 - Random Delay: `time.sleep(random.uniform(0.1, 0.3))` giữa các

request.

- Auto-Retry: Tự động thử lại 3 lần nếu gặp lỗi mạng hoặc Timeout.
- Xử lý mã lỗi 429 (Too Many Requests): Tự động ngủ 30 giây khi server phản hồi quá tải.

d. Tối ưu hóa Hiệu năng (Performance Optimization):

- Thread-Safe Writing: Sử dụng `threading.Lock()` để quản lý việc ghi file JSONL, cho phép nhiều luồng cùng ghi dữ liệu (Streaming Write) mà không gây lỗi race condition.
- Deferred Tokenization: Để tối đa hóa tốc độ thu thập, quá trình tách từ (Tokenization) được tách khỏi crawler và thực hiện ở giai đoạn hậu xử lý (Post-processing). Crawler chỉ thực hiện làm sạch cơ bản (Clean HTML, Unicode NFC).

2.2 Pipeline Xử lý Dữ liệu (Data Processing)

Dữ liệu thô (Raw Data) đi qua một quy trình làm sạch nghiêm ngặt gồm 4 bước để đảm bảo chất lượng cho Search Engine:

1. **Noise Removal (Loại nhiễu):**

- Sử dụng Regex loại bỏ thẻ HTML (`<script>`, `<div>`, `<a>`).
- Loại bỏ các thành phần giao diện thừa (Button text, Footer).

2. **Unicode Normalization:**

- Chuẩn hóa toàn bộ văn bản về bảng mã **Unicode NFC** (dùng sẵn) để thống nhất font chữ tiếng Việt.

3. **Tokenization & English Handling:**

- Sử dụng **PyVi** để tách từ tiếng Việt (ví dụ: xã_hội, công_nghệ).
- Xử lý tách biệt từ khóa Tiếng Anh dính liền dấu câu (ví dụ: Python, -> python).

4. **Quality Filter & Deduplication:**

- **De-duplication:** Loại bỏ trùng lặp dựa trên mã băm nội dung (MD5 Hash).
- **Min-length Filter:** Chỉ giữ lại các document có `word_count >= 50`.

Minh chứng: Ảnh chụp màn hình quá trình lọc cho thấy hệ thống hoạt động ổn định với hơn 1.1 triệu dòng sạch được lưu.

3. PHÂN TÍCH DỮ LIỆU (Data Insights)

3.1 Phân bố độ dài Document

Dữ liệu tập trung chủ yếu vào các bình luận có độ dài trung bình (50-200 từ), phù hợp với tính chất thảo luận trên diễn đàn.

Khoảng độ dài	Số lượng	Tỷ lệ phần trăm
---------------	----------	-----------------

50 - 100 từ	646,307	56.94%
101 - 200 từ	336,139	29.61%
201 - 500 từ	120,397	10.61%
501 - 1000 từ	25,780	2.27%
> 1000 từ	6,453	0.57%

3.2 Top Tác giả đóng góp (Key Opinion Leaders)

Hệ thống xác định được các tác giả tích cực nhất, đây là nguồn dữ liệu quan trọng cho tính năng "User Ranking" sau này.

1. **Masterchiefs:** 6,018 bài.
2. **Bing AI:** 5,037 bài (Bot tin tức tự động - cần lưu ý xử lý riêng trong M2).
3. **Phanh Blank 2:** 3,416 bài.
4. **xuantruong:** 2,691 bài.
5. **Kia Seltos:** 2,428 bài.

3.3 Từ vựng & Chất lượng nội dung

- **Kích thước từ vựng (Vocabulary):** ~109,949 từ duy nhất (trong mẫu 100k docs).
- **Top Tokens:** người, làm, đi, năm, tiền.
- **Lưu ý:** Xuất hiện một số artifact do quá trình crawl như click, expand (Top 1, 3, 4). Nhóm đã ghi nhận và sẽ bổ sung **Stopword List** chuyên biệt để loại bỏ các từ này trong Milestone 2 (Indexing).

4. LƯU TRỮ & CẤU TRÚC (Storage & Structure)

4.1 Định dạng lưu trữ

Dữ liệu được lưu dưới dạng **JSON Lines (.jsonl)**.

- **Ưu điểm:** Tối ưu cho Big Data, có thể đọc/ghi theo dòng (Streaming) mà không cần load toàn bộ 2GB dữ liệu vào RAM.

4.2 Schema dữ liệu

Mỗi dòng trong file là một object JSON với cấu trúc:

JSON

```
{
  "doc_id": "voz_t1055393_p2",
  "thread_id": "t1055393",
  "content": "Nội dung gốc chưa xử lý...",
  "text_clean": "nội_dung đã xử lý tách từ...",
  "author": "username",
  "source": "voz",
  "url": "https://voz.vn/t/...",
  "word_count": 125
}
```

4.3 Cấu trúc thư mục GitHub

Plaintext

Birds-search-engine-main/

```
├── ai_log.md          # Nhật ký làm việc với AI (Zero Tolerance)
├── requirements.txt    # Thư viện: aiohttp, pyvi, beautifulsoup4
├── docs/
│   ├── Milestone1_Report.md # Báo cáo này
│   └── statistics_report.md # File log thống kê chi tiết
├── data_sample/
│   └── sample.jsonl      # Mẫu dữ liệu (500 dòng)
├── src/
│   └── crawler/
│       ├── spider.py     # Crawler chính
│       ├── parser.py     # Module phân tích HTML
│       └── processor.py  # Module làm sạch dữ liệu
```

5. KẾT LUẬN & HƯỚNG PHÁT TRIỂN

Nhóm đã hoàn thành xuất sắc Milestone 1 với bộ dữ liệu lớn, sạch và được cấu trúc tốt.

- **Thành công:** Vượt chỉ tiêu số lượng (113%), quy trình làm sạch hiệu quả.
- **Cải thiện:** Trong Milestone 2 (Indexing), nhóm sẽ tập trung xây dựng bộ **Stopwords** để loại bỏ các từ rác kỹ thuật (click, expand) còn sót lại để tối ưu hóa chỉ mục đảo (Inverted Index).